

宁波大学
NINGBO UNIVERSITY

信息安全技术实验报告

[密码学综合设计实验]

姓名：	顾凯琪
学号：	226001624
课程名称：	信息安全技术
所在学院：	信息科学与工程学院
专业班级：	22 级计算机 2 班
任课教师：	徐博文 严迪群

实验项目名称	密码学综合设计实验

一、实验目的与要求：

安全通信是指两个用户在通信时不希望第三方监听。为此，他们需要以一种不易被窃听或拦截的方式进行交流。我们使用 CrypTool2.1 来模拟大致的过程，操作步骤如下：

1. 用户 A 通过 RSA 生成其公钥和私钥
2. 用户 B 通过 RSA 生成其公钥和私钥
3. 用户 A 创建一个文件（例如 message.txt 包含学号姓名信息）并输入一些字符串
4. 用户 A 为 DES 选择一个密钥（key）
5. 用户 A 使用 DES 用密钥 key 加密 message.txt 并获得密文（cipher.txt）
6. 用户 A 使用 RSA 用用户 B 的公钥加密密钥 key（key.txt）
7. 用户 A 使用 MD5 获取 message.txt 的哈希码（hash.txt）
8. 用户 A 通过 RSA 用用户 A 的私钥加密 hash.txt（signature.txt）
9. 用户 A 将 cipher.txt、key.txt、signature.txt 发送给用户 B
10. 用户 B 用他的私钥解密 key.txt 并获得密钥 key
11. 用户 B 使用密钥解密 cipher.txt 并获得 message.txt
12. 用户 B 使用用户 A 的公钥解密 signature.txt 并获得 hash.txt
13. 用户 B 使用 MD5 计算提取的 message.txt 的哈希码（hash_extracted.txt）
14. 用户 B 比较 hash.txt 和 hash_extracted.txt 之间的差异以检查消息的完整性
15. 任务结束

实验要求：

DES 密钥 key 为自己的学号

二、实验环境：

Windows10 系统
CrypTool 2.1 软件

三、实验方法（原理、流程图）

3.1 RSA 算法

1、原理

算法本身基于一个简单的数论知识：给出两个素数，很容易将它们相乘，然而给出它们的乘积，想得到这两个素数就显得尤为困难。如果能够解决大整数（比如几百位的整数）分解的快速方法，那么 RSA 算法将轻易被破解。

2、公钥和私钥的生成

RSA 算法由两个密钥，即公钥和私钥组成。

1）准备两个非常大的素数 p 和 q（转换成二进制后 1024 个二进制位或者更多，位数越多越难破解）；

2）利用字符串模拟计算大素数 p 和 q 的乘积 n=pq；

3）同样方法计算 m=(p-1)(q-1),这里的 m 为 n 的欧拉函数；

4）找到一个数 e(1<e<m)，满足 gcd(m,e)=1(即 e 和 m 互素)；

5）计算 e 在模 m 域上的逆元 d(满足 ed mod m=1)；

6）至此，公钥和私钥生成完毕：(n,e)为公钥，(n,d)为私钥;

3、RSA 加密

对于明文 x，用公钥(n,e)对 x 加密的过程，就是将 x 转换成数字(字符串的话取其 ASCII 码或者 unicode 值)，然后通过幂取模计算出 y，其中 y 就是密文；

$$y = x^e \ mod \ n$$

4、RSA 解密

对于密文 y，用私钥(n,d)对 y 进行解密的过程和加密类似，同样是计算幂取模：

$$x = y^d \ mod \ n$$

3.2 DES 加密

我们可以将 DES 加密过程分为以下三部分：

（1）明文的加密的整体过程

（2）F 轮函数解析

（3）密钥的形成过程

1.明文的加密整体过程

1） 输入 64bit 的明文进行 IP 置换，分成左右两个分支各为 32bit，左边：

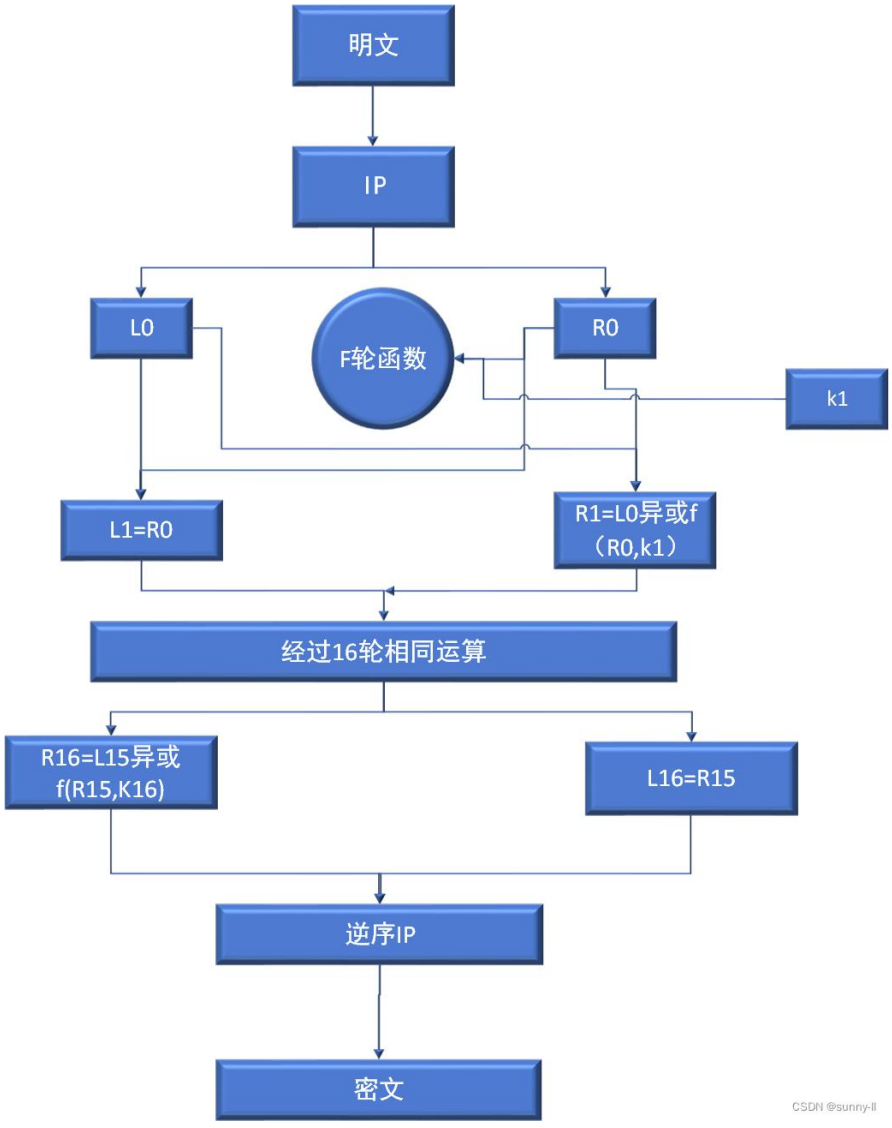
32bitL0,右边：32bitR0

2） 右分支：L1 = R0 左分支：引入 48bit 的密钥，R1=L0 异或 f(R0,K1)

3） 相同的操作进行 16 次的运算循环，算出相应的，R1~R16，L0~L16

4） 最后在进行 IP 的逆序置换，将左右两个分支再次合并为 64bit 密文

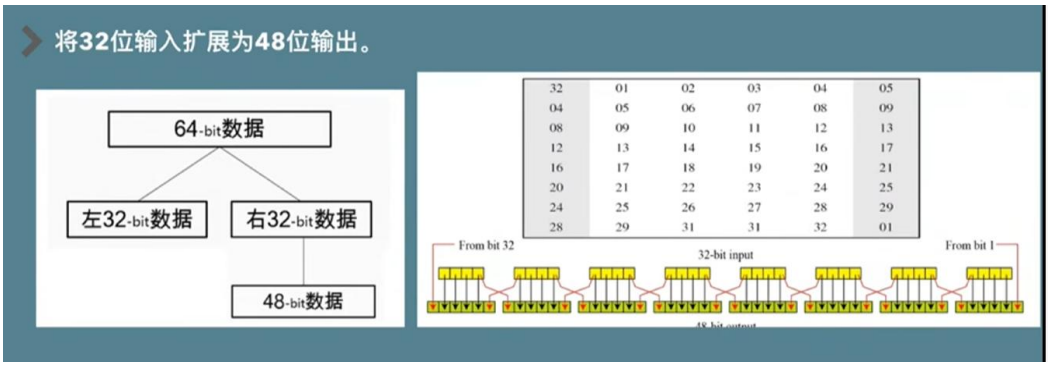
我们将上述过程画出流程图如下：



2、F 轮函数解析

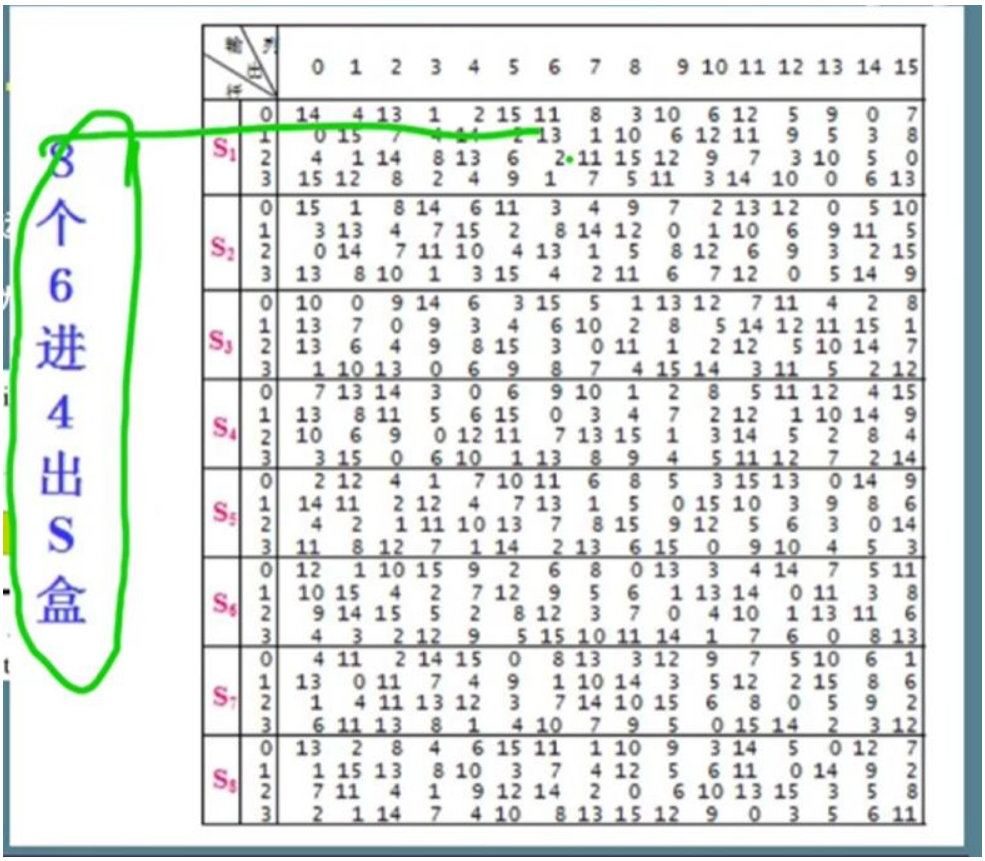
（1）IP 置换

（2）E 扩展：将 32bit 的 R0 扩展为 48bit 的 R0,其中扩展图为：



（3）异或：将 48bit 的 R0 于 48bit 的 K1 进行异或。

（4）S 盒压缩处理 ：大盒子里有 8 块 6bit 的小盒子，刚好容纳 48bit 的二进制数，盒子的特点是 6 进 4 出，出了盒子就变成了 32bit 的二进制数。



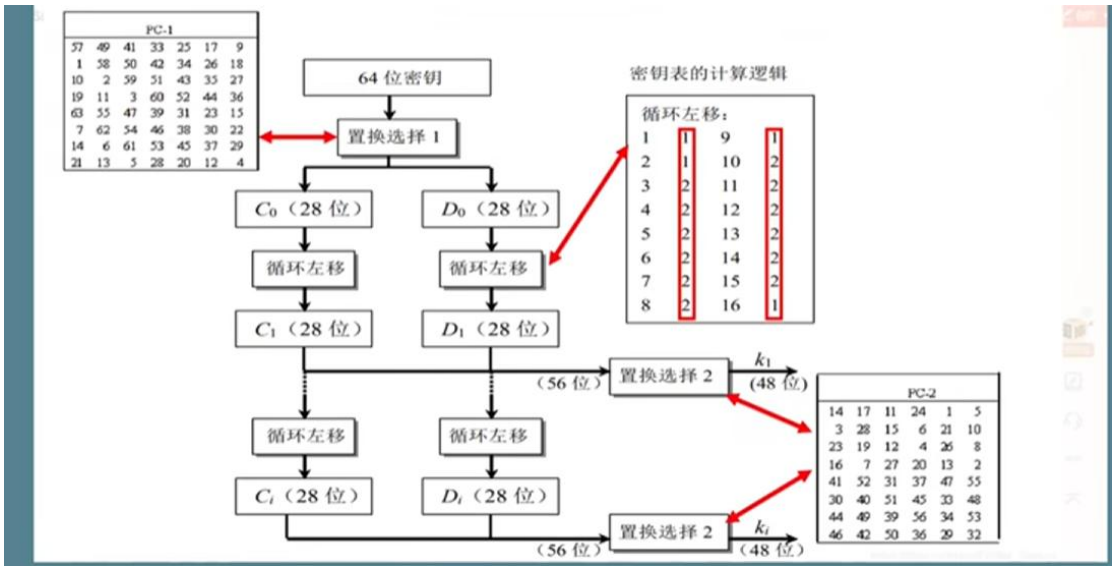
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S_1	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

（5）IP 逆序置换

3.密钥的形成过程

（1）密钥原本为 64bit ，去掉 8 位校验位，剩余 56 位参与运算。

（2）按照交换规则，生成 16 位 48bit 的轮密钥。



3.3 MD5 算法

MD5（单向散列算法） 的全称是 Message-Digest Algorithm 5（信息-摘要算法），经 MD2、MD3 和 MD4 发展而来。MD5 算法的使用不需要支付任何版权费用。

MD5 功能：

一、输入任意长度的信息，经过处理，输出为 128 位的信息（数字指纹）；

二、不同的输入得到的不同的结果（唯一性）；

三、根据 128 位的输出结果不可能反推出输入的信息（不可逆）；

MD5 用途：

1、 防止被篡改 ：

1）比如发送一个电子文档，发送前，我先得到 MD5 的输出结果 a。然后在对方收到电子文档后，对方也得到一个 MD5 的输出结果 b。如果 a 与 b 一样就代表中途未被篡改。、

2）比如我提供文件下载，为了防止不法分子在安装程序中添加木马，我可以在网站上公布由安装文件得到的 MD5 输出结果。

3）SVN 在检测文件是否在 CheckOut 后被修改过，也是用到了 MD5.

2、 防止直接看到明文 ：

现在很多网站在数据库存储用户的密码的时候都是存储用户密码的 MD5 值。这样就算不法分子得到数据库的用户密码的 MD5 值，也无法知道用户的密码(其实这样是不安全的，后面我会提到)。(比如在 UNIX 系统中用户的密码就是以 MD5（或其它类似的算法）经加密后

存储在文件系统中。当用户登录的时候，系统把用户输入的密码计算成MD5值，然后再去和保存在文件系统中的MD5值进行比较，进而确定输入的密码是否正确。通过这样的步骤，系统在并不知道用户密码的明码的情况下就可以确定用户登录系统的合法性。这不但可以避免用户的密码被具有系统管理员权限的用户知道，而且还在一定程度上增加了密码被破解的难度。）

3、 防止抵赖（数字签名）：

这需要一个第三方认证机构。例如A写了一个文件，认证机构对此文件用MD5算法产生摘要信息并做好记录。若以后A说这文件不是他写的，权威机构只需对此文件重新产生摘要信息，然后跟记录在册的摘要信息进行比对，相同的话，就证明是A写的了。这就是所谓的“数字签名”。

MD5算法过程：

对MD5算法简要的叙述可以为：MD5以512位分组来处理输入的信息，且每一分组又被划分为16个32位子分组，经过了一系列的处理后，算法的输出由四个32位分组组成，将这四个32位分组级联后将生成一个128位散列值。

一、填充：如果输入信息的长度(bit)对512求余的结果不等于448，就需要填充使得对512求余的结果等于448。填充的方法是填充一个1和n个0。填充完后，信息的长度就为N*512+448(bit)；

二、记录信息长度：用64位来存储填充前信息长度。这64位加在第一步结果的后面，这样信息长度就变为N*512+448+64=(N+1)*512位。

三、装入标准的幻数（四个整数）：标准的幻数（物理顺序）是（A=(01234567)$_{16}$，B=(89ABCDEF)$_{16}$，C=(FEDCBA98)$_{16}$，D=(76543210)$_{16}$）。如果在程序中定义应该是（A=0X67452301L，B=0XEFCDAB89L，C=0X98BADCFEL，D=0X10325476L）。有点晕哈，其实想一想就明白了。

四、四轮循环运算：循环的次数是分组的个数（N+1）

四、实验过程、步骤及内容

4.1 RSA 算法为非对称加密算法，需要生成不同用户的公钥和私钥。

4.2 我们选择我们的学号作为用户 A 使用 DES 加密算法的密钥 key，并使用该密钥 key 来加密我们想要传输的信息 message.txt，获得密文 cipher.txt，并将密文 cipher.txt 发送给用户 B。

4.3 接下来为了让 DES 的密钥能够顺利传递给 B，用户 A 使用 RSA 算法用用户 B 的公钥来加密密钥 key，得到 key.txt，并将 key.txt 发送给用户 B。

4.4 为了判断中途是否会受到中间人攻击而篡改文件信息，用户 A 还是用 MD5 算法得到了 message.txt 的哈希码 hash.txt。

4.5 为了传递 hash.txt，用户 A 使用自己的私钥加密了 hash.txt 文件为 signature.txt 文件。这样做的目的能够产生一个数字签名的效果，是外部通信的人明确这个消息时用户 A 所发送的。

4.6 用户 B 在接收到密文 cipher.txt、加密后的密钥 key.txt 和加密后的 hash 值 signature.txt 后。首先采用 RSA 算法使用自己的密钥来破解 key.txt 从而获取 DES 算法的 key。

4.7 接下来用户 B 使用破解得到的 key 来破解接收得到的密文 cipher.txt，得到传送的明文 message.txt。

4.8 用户 B 使用用户 A 的公钥对用户 A 私钥加密过的 hash 密文 signature.txt 进行解密，得到传递过来的 hash.txt。

4.9 对破解得到的明文 message.txt 计算哈希值，得到 hash_extracted.txt 的的哈希码，将这个哈希值与传递来的哈希值 hash.txt 进行比较，观察传递的信息文件是否被篡改以检查消息的完整性。

五、实验数据（现象）处理分析

该部分包含实验的主要操作过程与实验现象的记录。

5.1 消息传输过程分析

根据实验要求，分析 A、B 用户之间的数据处理流程，为了方便理解加解密过程，同时便于后续 Crytools 的操作，对消息传输的过程进行绘图分析。整个消息传输过程如图 5.1 所示。

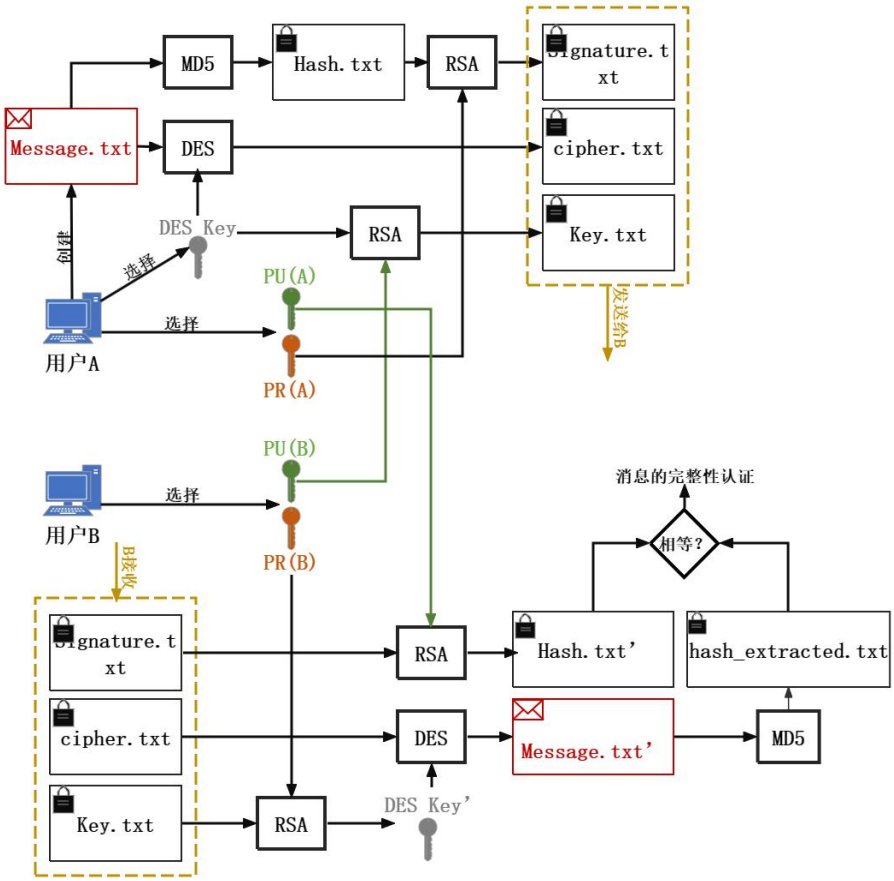


图 5.1 消息传输流程图

分析以上流程图可知，整个消息传输过程可以大致分为两个部分，分别 A、B 用户的操作。在所有操作之前，双方都要进行 RSA 密钥的选定，这里双方的 RSA 公私钥不一定相同。

对于 A 用户，需要选定用于 DES 加密的密钥 Key，对于待发送消息 Message，使用 DES 进行加密得到密文 Cipher，该密文需要发送给用户 B，DES 的密钥 Key 也需要进行发送；但为了保证密钥安全，使用 RSA 对 DES 的密钥进行加密，为了使得 Key 只有 B 知道，需要使用 B 的 RSA 公钥进行加密；为了使 B 能够对消息 Message 进行完整性验证，A 使用 MD5 对 Message 进行哈希提取，得到的结果一并发送给 B，但为了不受到非法篡改，需要对该哈希结果 Hash

进行数字签名，即使用 A 用户的 RSA 私钥进行加密，得到签名后的哈希结果 Signature。

对于用户 B，接收到经过 DES 加密的密文 Cipher、DES 密钥的 RSA 加密结果、数字签名后的 Message 哈希。首先为了获取 DES 的密钥，B 使用自己的 RSA 私钥即可进行解密，并对密文 Cipher 进行 DES 解密即可得到解密后的明文，但为了判定该明文的可信程度，对其进行 Hash 提取，并使用 A 的公钥对 Signature 进行解密，得到来自于 A 的 Hash，比较两个 Hash 结果即可判定消息的完整性，若密文受到攻击则会导致 B 提取的 Hash 有所偏差，若 A 的 Signature 受到攻击则会导致认证结果（RSA 解密后的来自 A 的 Hash）错误。

5.2 使用 Graphical Editor 实现以上消息加密传输

根据图 5.1，能够容易地使用 Graphical Editor 实现该通信过程。

（1）RSA 密钥生成

对于通信双方的 RSA 密钥，生成方式依照 3.1 进行，但由于数据量较大，需要使用数值较大的素数，这将带来较大的计算量，同时为了保证实验过程的可信程度，本次实验使用 Cryptool 中的 RSA 密钥随机生成功能进行密钥的生成。并将产生的密钥值（公共大数 N，公开的 d，私有的 e，A、B 各一组）存储到变量中以便后续的数据调用。如图 5.2 所示。

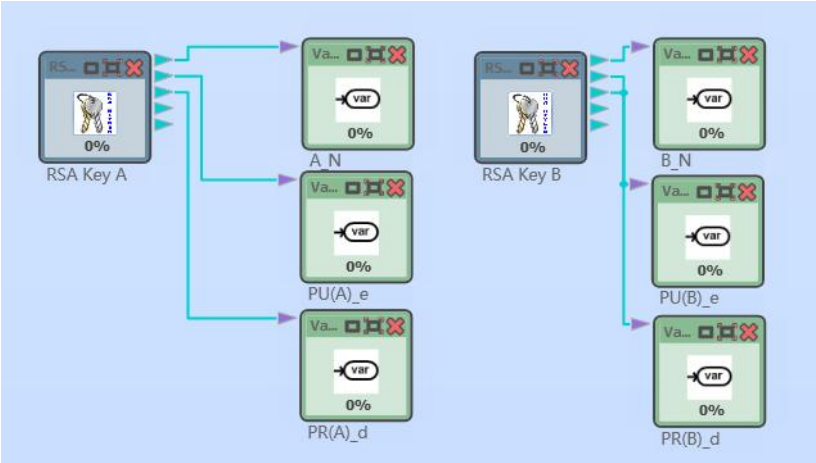


图 5.2 RSA 密钥生成

（2）用户 A 操作实现

对于用户 A 的操作，包括使用 DES 对明文进行加密得到密文、对 DES 密钥进行 RSA 加密、使用 MD5 对明文进行 Hash 提取并使用 RSA 数字签名。生成的三部分内容将发送给用户 B。实现过程如图 5.3 所示。

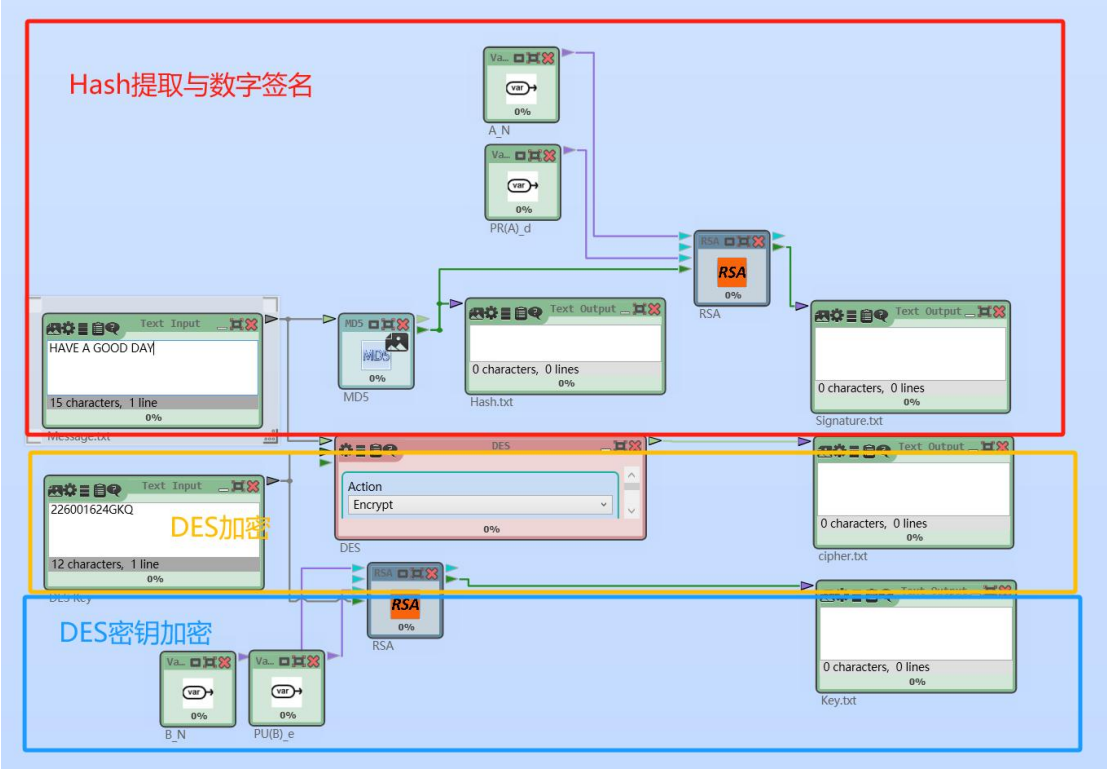


图 5.3 基于 Graphical Editor 的用户 A 操作实现

（3）用户 B 操作实现

对于用户 B 的操作，包括使用 RSA 对 DES 密钥进行解密、使用解密得到的 DES 密钥解密密文得到明文、解密数字签名后的 Hash、对解密得到的明文进行 Hash 提取、比较两个 Hash 值以验证完整性。实现过程如图 5.4 所示。

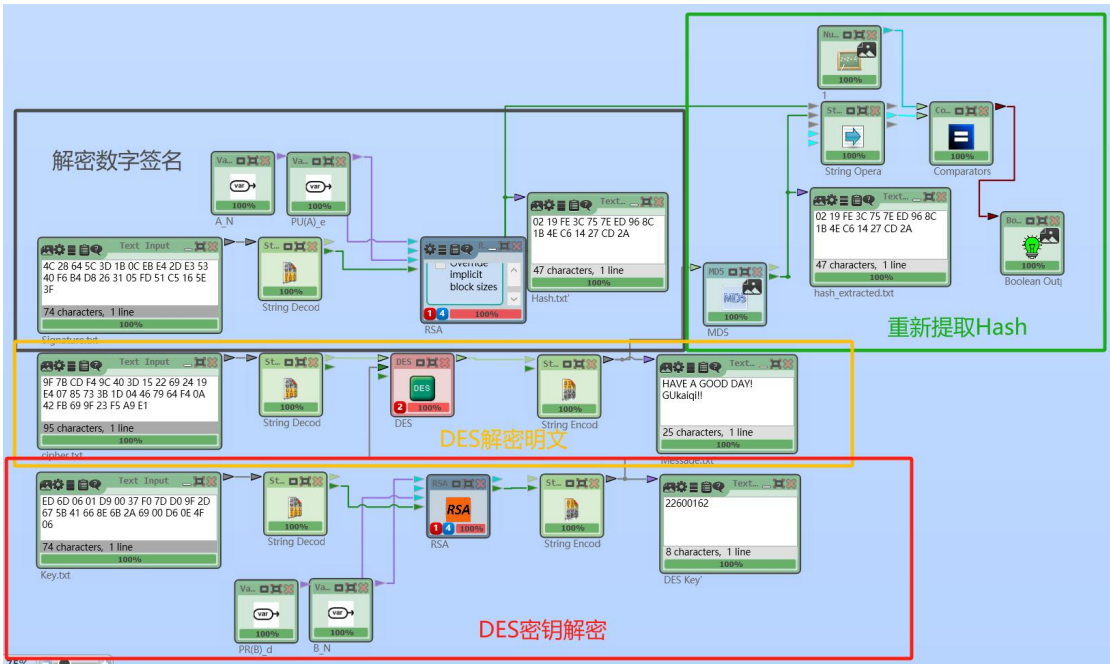


图 5.4 基于 Graphical Editor 的用户 B 操作实现

5.3 加解密测试

对以上实现的内容进行测试。

对于用户 A，选定 DES 的密钥为“22600162”，共 8 字节长，符合 DES 的密钥长度需求，待传输信息为“HAVE A GOOD DAY!GUkaiqi!!”。A 用户的操作结果如图 5.5 所示。对于明文的 MD5 提取结果为 128 位（16*8）的散列值，对应的 16 进制数值为“02 19 FE 3C 75 7E ED 96 8C 1B 4E C6 14 27 CD 2A”。

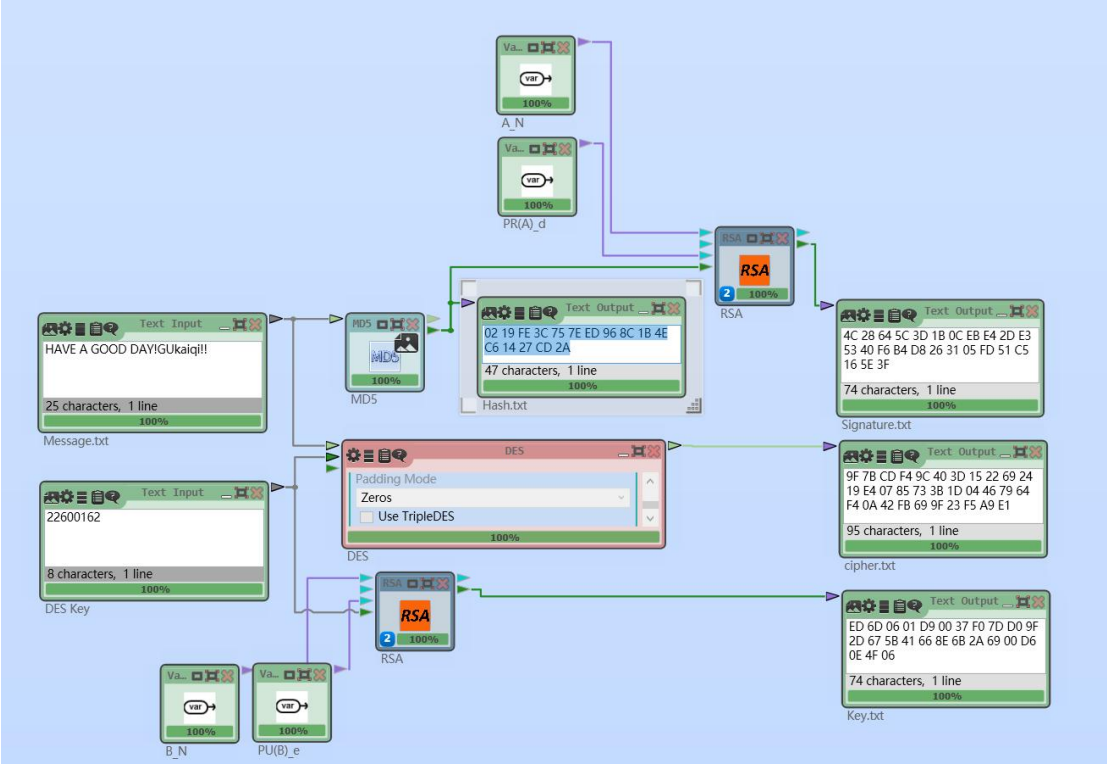


图 5.5 用户 A 的操作结果

假设用户 B 能够正确地接收到用户 A 的三份文本（Key、Cipher、Signature），则能够顺利进行解密。如图 5.6 所示，用户 B 解密得到的 DES 密钥正确，基于 DES 密钥解密得到的明文也正确，最终解密认证得到的 Hash 值与重新提取的 Hash 值相同，由此可以判定传输没有受到攻击，同时能够保证其完整性。

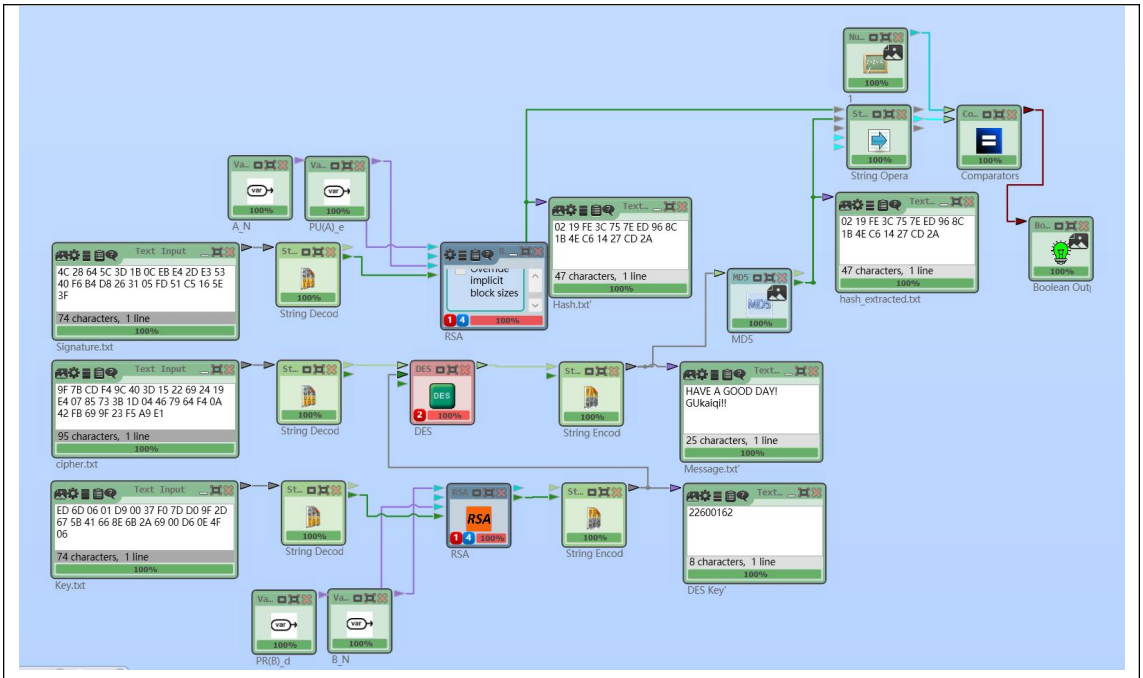


图 5.6 传输正确情况下的解密

5.4 模拟攻击

接下来实验对以上传输过程进行模拟攻击，首先若用户 A 加密的 DES 密钥受到篡改，则无法解密得到正确的 DES 密钥，更无法得到明文。如图 5.7 所示，由此可以看出若该部分信息受到拦截或者恶意篡改，就会影响到最终的解密。

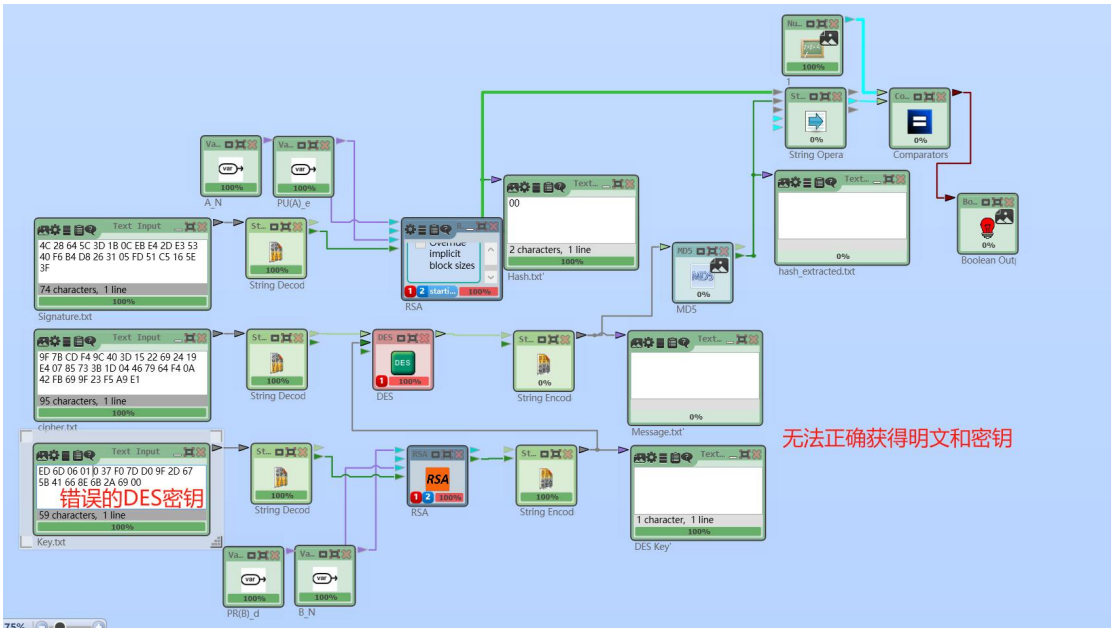


图 5.7 加密 DES 密钥受到篡改

若密文 Cipher 受到篡改，则即使得到了正确的 DES 密钥，也无法解密得到正确的明文，更无法进行后续的 Hash 提取，故无法通过认证，保证了一定的安全性，如图 5.8 所示。

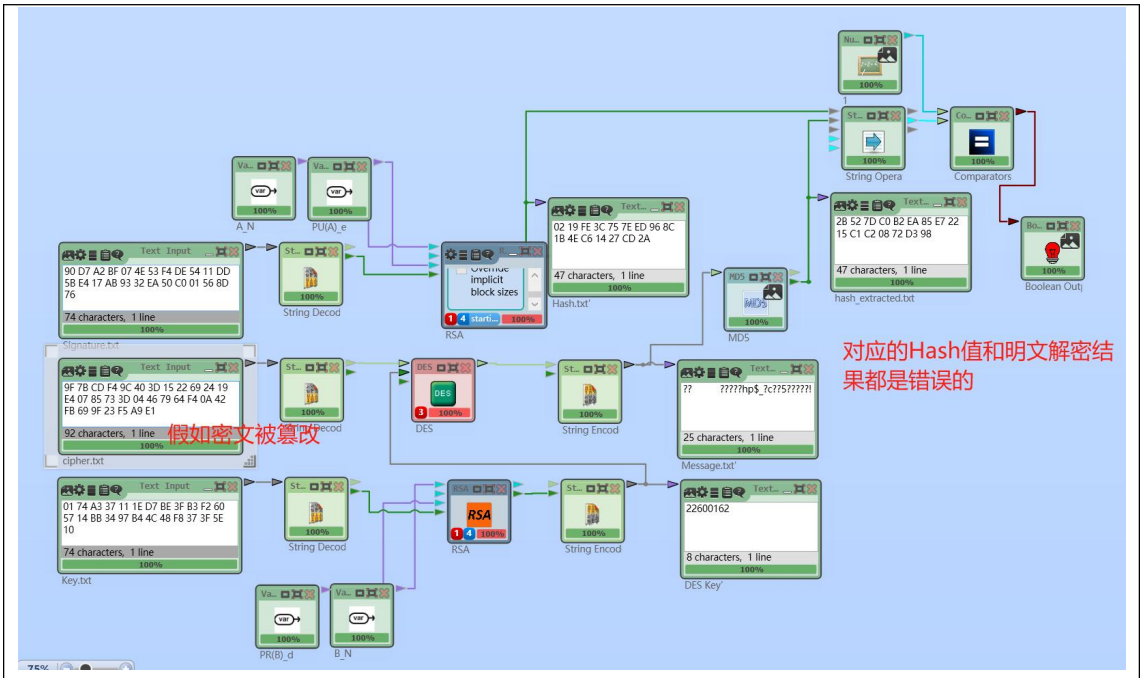


图 5.8 密文受到篡改

若经过数字签名后的 Hash 值 Signature 受到了篡改，则无法经过 RSA 解密得到正确的来自 A 的 Hash 值，即使其余消息接收无误，最终提取的 Hash 值必定与之不同，则无法通过最终的完整性认证，如图 5.9 所示。

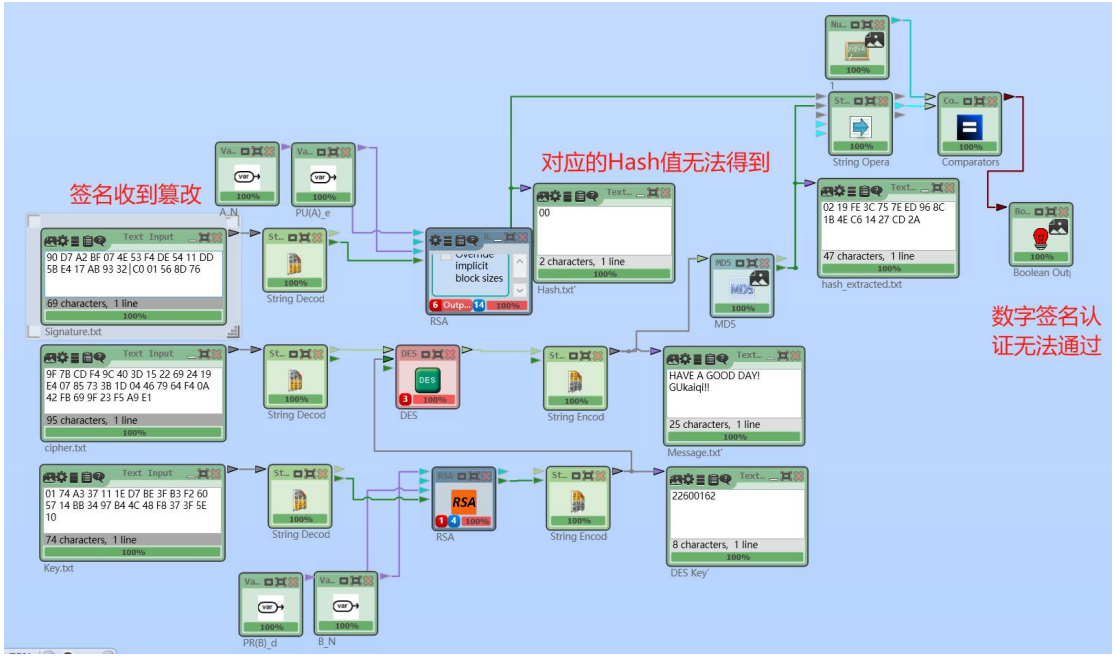


图 5.9 签名后的 Hash 值受到篡改

综上，若传输的各类数据中有其一受到了篡改，最终都无法得到正确的结果，这可能体现在无法解密得到明文（可能是无法理解的明文），也可能体现在无法通过完整性认证，因此以上传输操作在一定程度上提供了一种安全通信的方案，即受到篡改的通信对于通信的

接收方来说是可识别的。

与此同时，对于解密的关键信息，例如 DES 的密钥，通过了非对称加密方案进行加密，在一定程度上也防止了被窃听或者拦截的可能性。

六、实验结论

通过本次密码学综合设计实验，我们成功地模拟了安全通信的过程，并且验证了非对称加密、对称加密以及数字签名在保障信息传输安全中的重要性和有效性。实验结果表明，所采用的加密和验证方法能够达到预期的安全目标。

1. **非对称加密（RSA）的有效性**：通过使用 RSA 算法生成公钥和私钥，我们实现了密钥的安全交换。用户 A 利用用户 B 的公钥加密 DES 算法的密钥，确保了密钥在传输过程中的安全性。
2. **对称加密（DES）的应用**：DES 算法的使用展现了对称加密在数据加密方面的高效性。我们选择学号作为密钥，不仅因为其唯一性，也因为它与用户个人直接相关，便于记忆和管理。
3. **数字签名（MD5）的作用**：通过 MD5 算法生成的哈希值和随后的数字签名过程，我们确保了信息的完整性和不可否认性。用户 B 通过比较收到的哈希值和自行计算的哈希值，验证了信息在传输过程中未被篡改。
4. **实验流程的合理性**：实验步骤清晰，操作顺序合理，从密钥生成到信息的最终验证，每一步都为信息安全提供了必要的保障。
5. **安全性的验证**：实验中，用户 B 成功解密了密文，验证了数字签名，并确认了信息的完整性。这一过程展示了整个通信过程中的安全措施的有效性。
6. **实验局限性**：尽管实验取得了成功，但也应注意到 DES 算法和 MD5 算法在当前的安

全标准下已不被认为是最安全的加密和哈希算法。在实际应用中，应考虑使用更先进的加密标准，如 AES 和 SHA-256。

7. **未来展望**：本次实验为理解密码学在信息安全中的作用提供了实践基础。未来的工作可以探索更复杂的密码学技术，如量子加密，以及它们在不同通信场景下的应用。

综上所述，本实验不仅加深了对密码学基本概念的理解，而且提高了我们对实际应用中信息安全重要性的认识。通过本次实验，我们学会了如何综合运用不同的密码学工具来保护信息的安全。

七、教师批阅意见：

成绩评定：

教师签字：

年　　月　　日

八、备注：